



Data Structures

Lecture 8: Hash Tables

Prof. Wang Haiyan
School of Computer Science
wanghy@njupt.edu.cn

Keys versus Indexes

Reminder:

- A dictionary contains key/datum pairs
- A key is not an index

Keys versus Indexes

Reminder:

- A dictionary contains key/datum pairs
- A key is not an index
- Retrieval by key means:
“Get me the record with key Smith”

Keys versus Indexes

Reminder:

- A dictionary contains key/datum pairs
- A key is not an index
- Retrieval by key means:
“Get me the record with key Smith”
- Retrieval by index means:
“Get me the 15th record”
- (If the data is sorted by key, this also means “get me the record with the 15th smallest key”)

Keys versus Indexes

Reminder:

- A dictionary contains key/datum pairs
- A key is not an index
- Retrieval by key means:
“Get me the record with key Smith”
- Retrieval by index means:
“Get me the 15th record”
- (If the data is sorted by key, this also means “get me the record with the 15th smallest key”)
- Retrieval by index can be achieved in constant, $O(1)$, time using an array
- Implementing a dictionary requires retrieval by key
- The best implementation we’ve seen so far is a balanced binary search tree allowing worst case retrieval by key in $O(\log n)$ time
- Insert/delete/successor/predecessor are also all $O(\log n)$ time

Today's Topic

Today's topic is hash tables

The idea is to **replace searching for a key** with a way to **turn a key directly into an index**

If we could do this, we could achieve retrieval by key (and perhaps other operations) in $O(1)$ time

Today's Topic

Today's topic is hash tables

The idea is to **replace searching for a key** with a way to **turn a key directly into an index**

If we could do this, we could achieve retrieval by key (and perhaps other operations) in $O(1)$ time

Note:

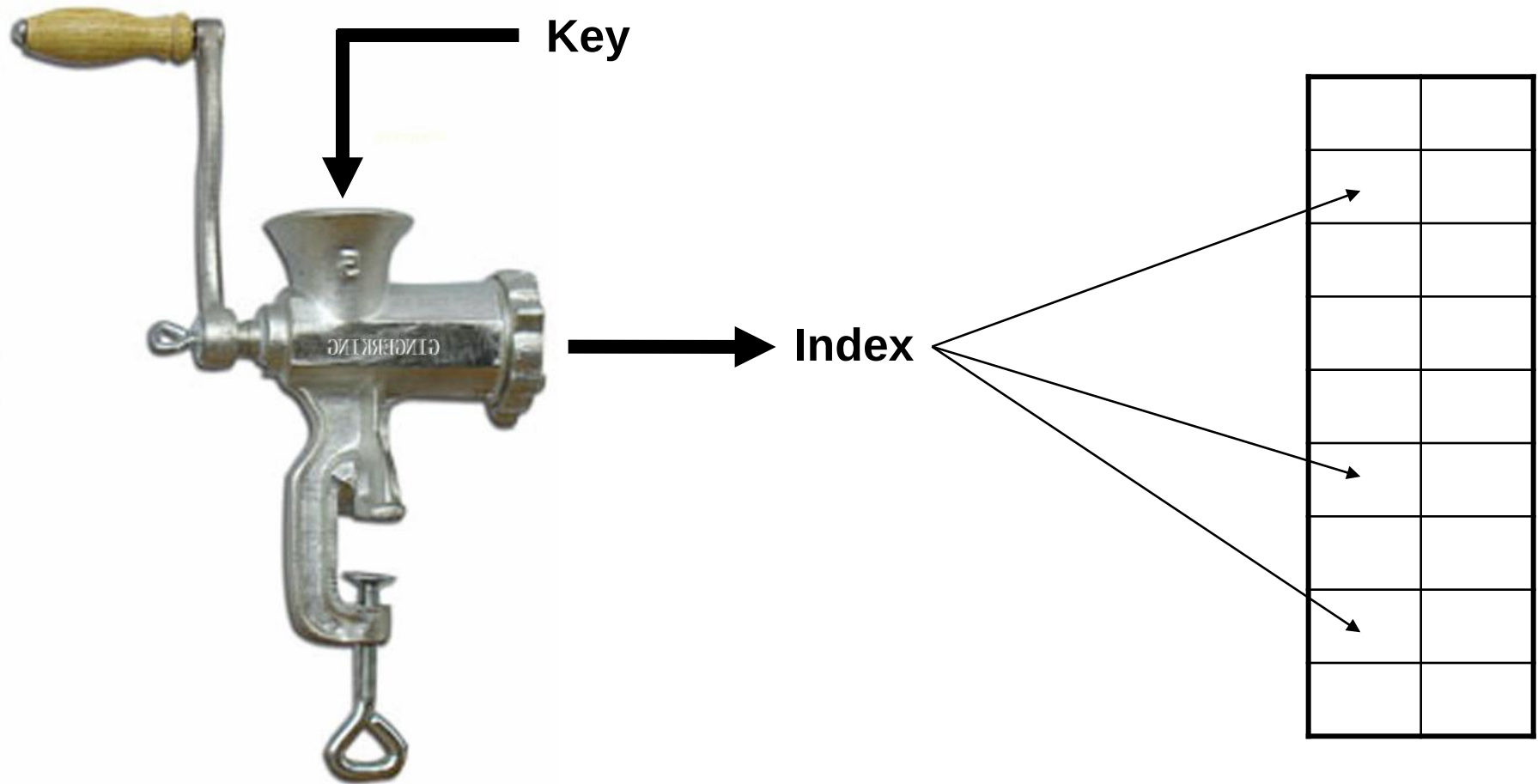
Hash tables are messy and inelegant

They give useful performance only in the average case and are guaranteed to break down

But they are very practical and are the right implementation choice for many problems

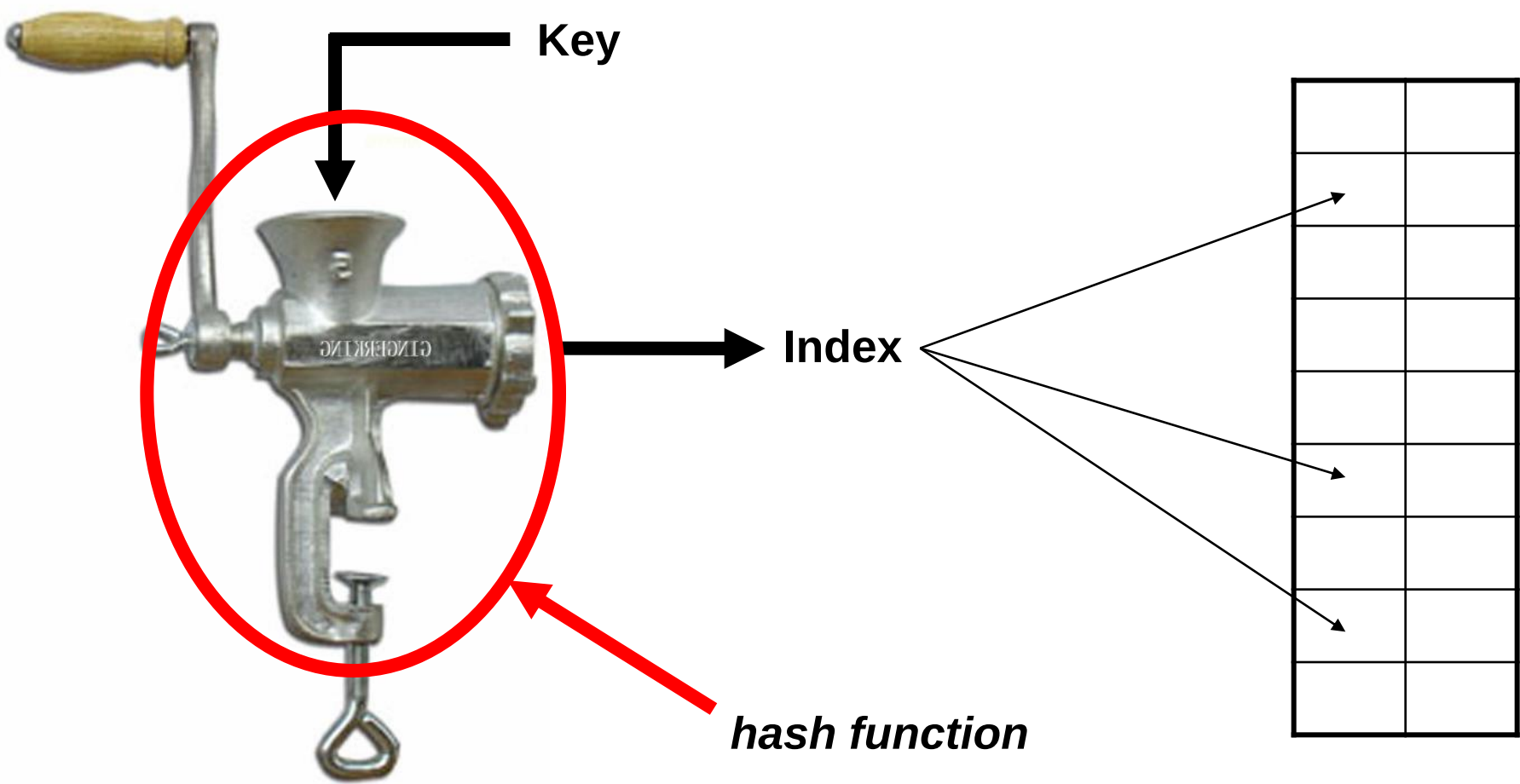
Overview of Hashing

Let's suppose we have a magic tool that turns keys into index values:



Overview of Hashing

Let's suppose we have a magic tool that turns keys into index values:



Overview of Hashing

Setting aside several practical issues...

...if we had such a hash function, h , we could get constant time insert and retrieve operations by turning a key into an index which we would use to address an array

To insert: pass key to hash function, insert datum into array index returned by hash function

To retrieve: pass key to hash function, read datum from array index returned by hash function

We know that accessing an array by index is $O(1)$, so assuming hash function is also $O(1)$, then so are insert and retrieve

Overview of Hashing

This exactly the process used by ***associative arrays***

In Perl (and others) you can say things like:

```
telephone[ 'smith' ] := "01904 434733"
```

i.e. we use a key to index an array

This is a neat programming construct, but it's not obvious how it would be implemented

This is exactly what we'll see today

Direct-address tables

Q: Suggest a simple way to turn keys into indexes.

Direct-address tables

Q: Suggest a simple way to turn keys into indexes.

A: Use the key as the index.

E.g. key drawn from universe:

$$U = \{1, 2, \dots, m\}$$

where m is not too large

We simply use an array of size m :

$$A[1..m]$$

Direct-address tables

Q: Suggest a simple way to turn keys into indexes.

A: Use the key as the index.

E.g. key drawn from universe:

$$U = \{1, 2, \dots, m\}$$

where m is not too large

We simply use an array of size m :

$$A[1..m]$$

Dictionary operations are then trivial:

```
Lookup(A, k)
    return A[k]
```

```
Insert(A, e)
    A[key(e)] := e
```

```
Delete(A, k)
    A[k] := NULL
```

} All operations are $O(1)$

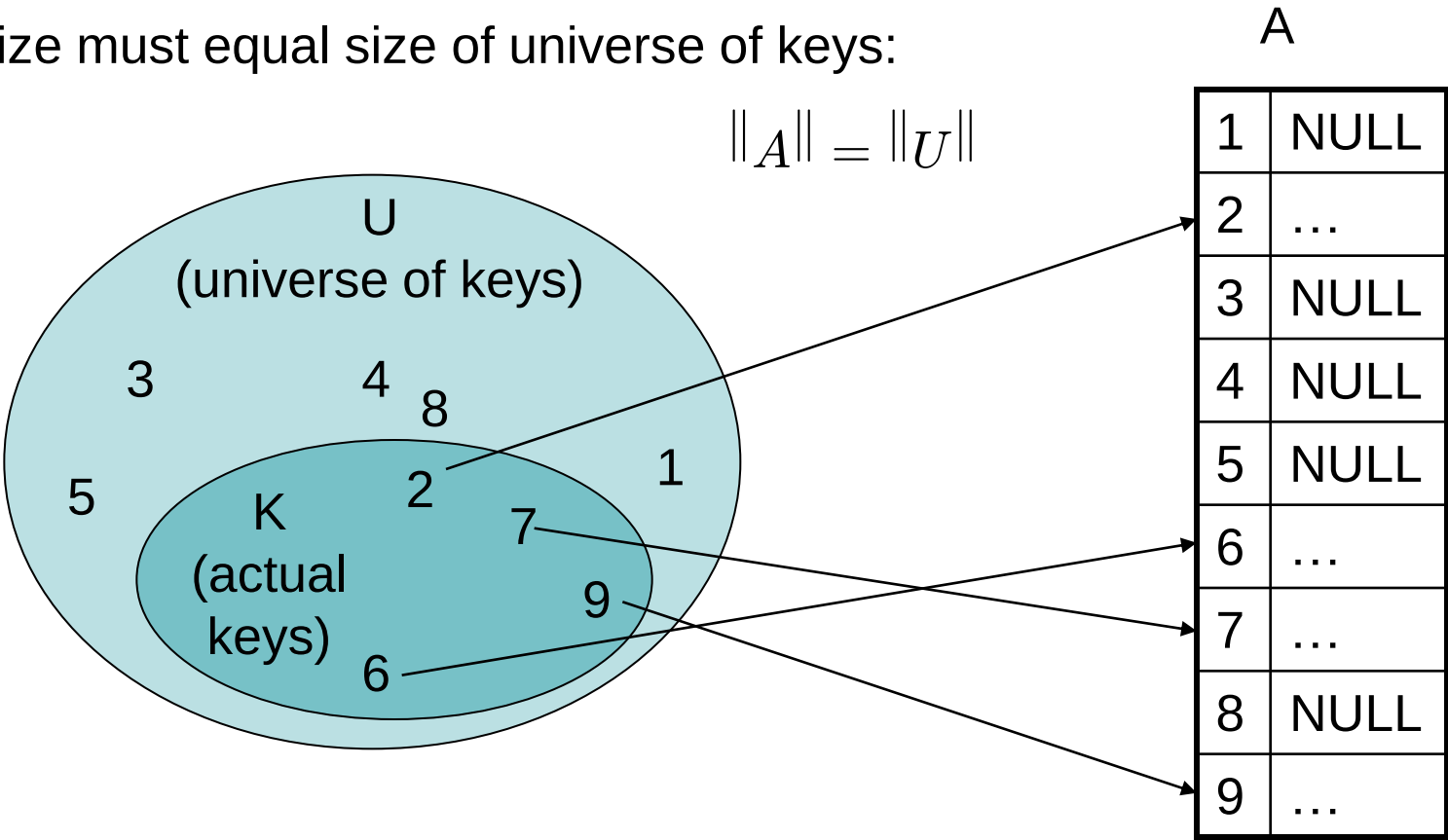
Q: Any problems with this approach?

Direct-address tables

Function from keys to indexes is one to one

Array size must equal size of universe of keys:

$$\|A\| = \|U\|$$

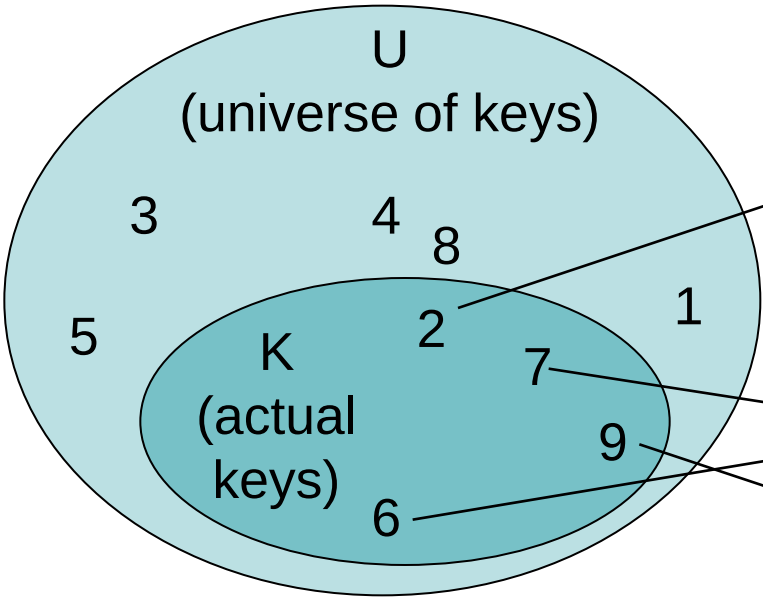


Direct-address tables

Function from keys to indexes is one to one

Array size must equal size of universe of keys:

$$\|A\| = \|U\|$$



A

1	NULL
2	...
3	NULL
4	NULL
5	NULL
6	...
7	...
8	NULL
9	...

Size of U may be huge

Imagine a key that can be any 64-bit number: would need an array of size 18,446,744,073,709,551,616

Hash Functions

When the actual number of keys stored will be much less than the size of U , a direct address table is impractical

Consider a standard dictionary of $\sim 100,000$ words

100,000 keys in use versus every possible combination of each of the 26 letters up to the length of the longest word – this is astronomical!

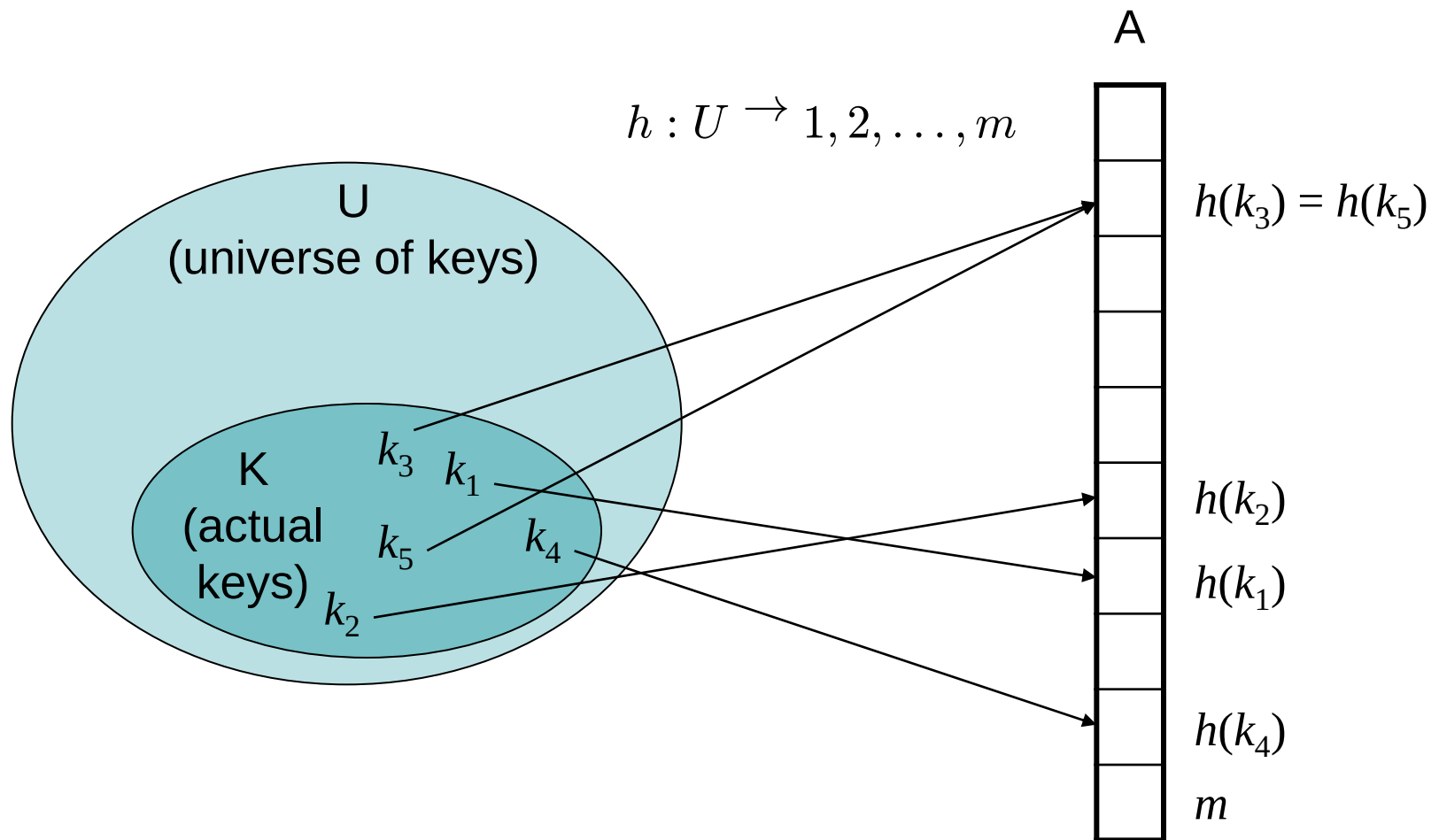
We would like the storage requirements to be $\Theta(|K|)$, i.e. only allocate space we actually need in practice

Instead of storing key k in slot k , with a hash function we store it in slot $h(k)$:

$$h : U \rightarrow 1, 2, \dots, m$$

We say key k **hashes** to slot $h(k)$ and $h(k)$ is the **hash value** of k

Hash Functions



Collisions

Since our hash function is many to one, inevitably different keys will hash to the same slot

When we try to insert an element into our hash table and find that the slot already contains an element, this is called a ***collision***

We must always have a strategy for dealing with collisions, even if we have a large array

Q: How many people do I need in a room to have a 50% chance that two of them share a birthday?

Collisions

Since our hash function is many to one, inevitably different keys will hash to the same slot

When we try to insert an element into our hash table and find that the slot already contains an element, this is called a ***collision***

We must always have a strategy for dealing with collisions, even if we have a large array

Q: How many people do I need in a room to have a 50% chance that two of them share a birthday?

A: 23 (assuming any day of the year is an equally likely birthday)

Collisions

Since our hash function is many to one, inevitably different keys will hash to the same slot

When we try to insert an element into our hash table and find that the slot already contains an element, this is called a ***collision***

We must always have a strategy for dealing with collisions, even if we have a large array

Q: How many people do I need in a room to have a 50% chance that two of them share a birthday?

A: 23 (assuming any day of the year is an equally likely birthday)

The lesson from this surprising result extends to collisions:

Even with a hash function that distributes keys evenly over a 1 million element array, there is a 95% chance of a collision before we've inserted 2500 elements

Collisions

Although collisions are inevitable, we still want to minimise them where possible

A good hash function will distribute keys seemingly randomly (though deterministically) across the range $1..m$

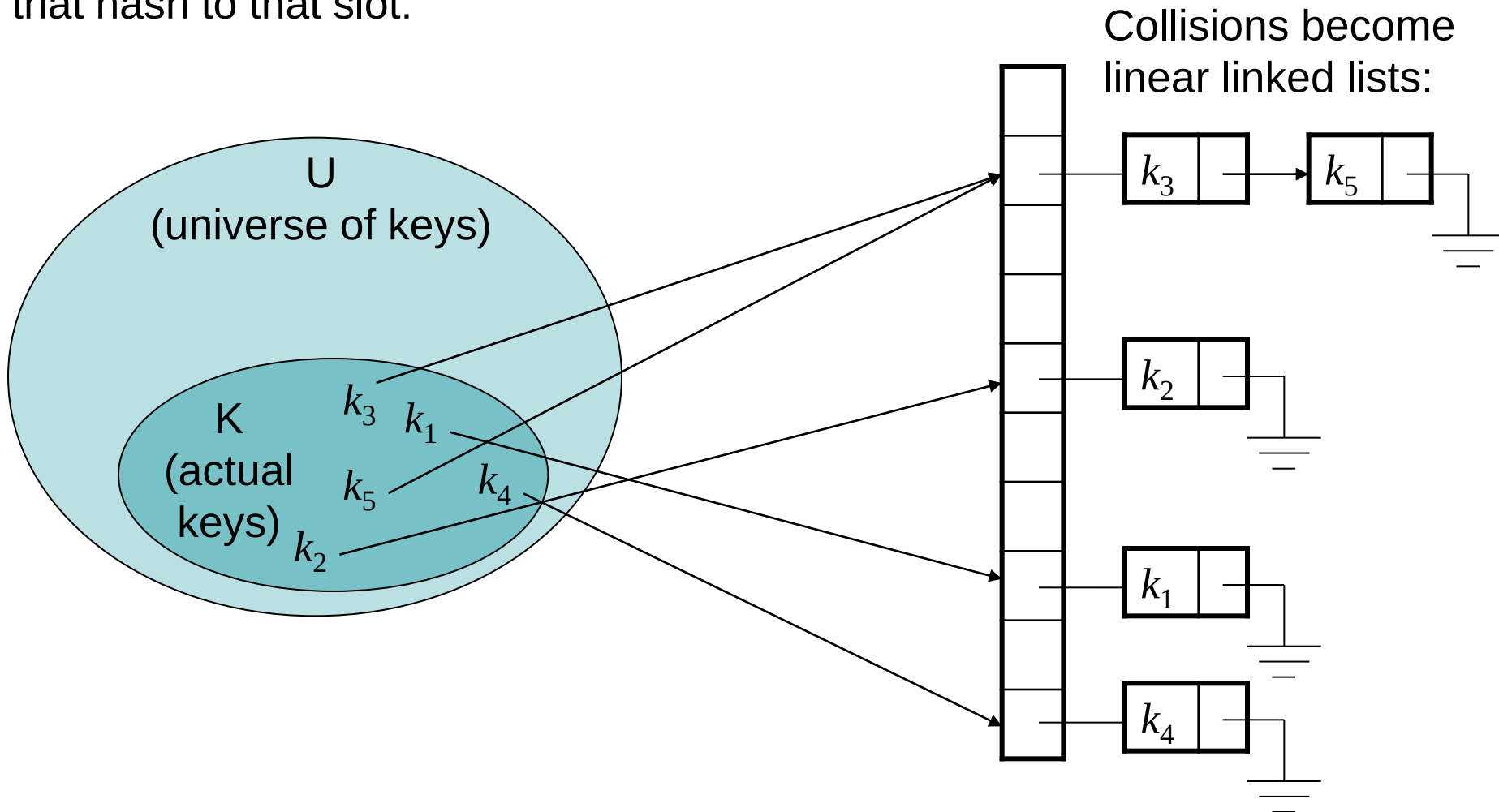
There are two approaches to tackling collisions in hashing:

Hashing with chaining (Open hashing)

Closed hashing

Hashing with Chaining

In hashing with chaining each slot becomes a list of all the elements that hash to that slot:



Hashing with Chaining

Q: What is complexity of inserting an item into such a hash table?

Hashing with Chaining

Q: What is complexity of inserting an item into such a hash table?

A: Assuming the linked lists are unsorted, we can add items to the front, so insertion is always $O(1)$.

Hashing with Chaining

Q: What is complexity of inserting an item into such a hash table?

A: Assuming the linked lists are unsorted, we can add items to the front, so insertion is always $O(1)$.

Q: Worst case time complexity of retrieval by key if we currently have n items in our dictionary?

Analysis of Hashing with Chaining

In the worst case, our hash function would map all keys to the same slot

Hash table would degenerate to linked list, retrieval would become $O(n)$

As mentioned at the beginning – hash tables are not generally used for their performance guarantees, but for their average performance

The result is good performance in practice

Let's look at average performance...

Analysis of Hashing with Chaining

Make assumption of *simple uniform hashing*:

- Each key k is equally likely to be hashed to any slot, independent of where other keys are hashed

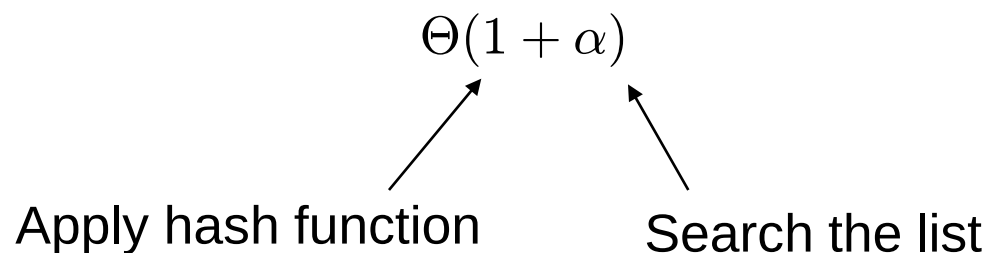
If there are currently n items in the hash table and m total slots, we can define the *load factor* of the table:

$$\alpha = \frac{n}{m}$$

= average number of keys per slot

Analysis of Hashing with Chaining

Worst case is when we are searching for an item that isn't there – we have to scan to end of list giving complexity:



So expected search time = $\Theta(1)$ if $\alpha = O(1)$

This happens when $n = O(m)$

Conclusion: if the number of hash table slots is at least proportional to the number of elements in the hash table, then average case retrieval is $O(1)$

A hash table with chaining supports all the dictionary operations in either worst or average case time $O(1)$

Hashing with Chaining

Exercise: insert the following keys into a hash table with chaining:

9, 2, 4, 7, 12

$$h(k) = k \bmod 5$$

Using the hash function:

And storing the head of the lists in the following array:

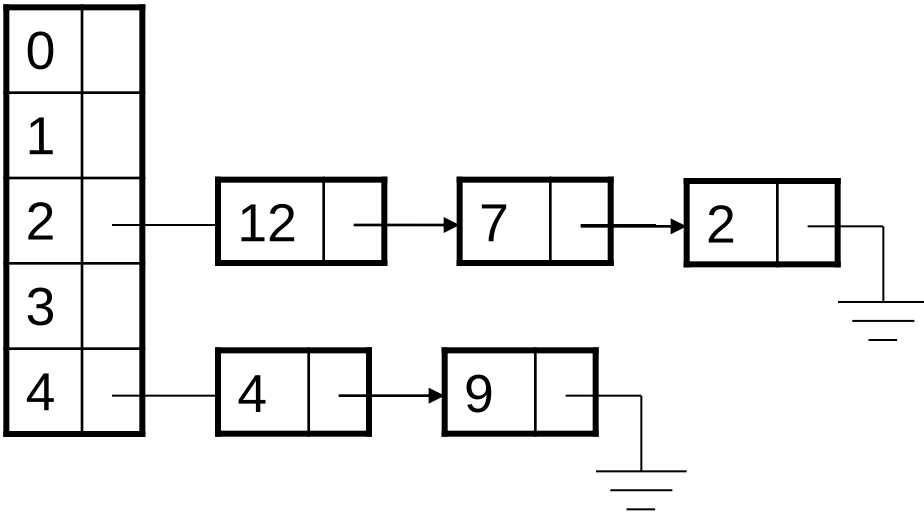
0	
1	
2	
3	
4	

Q: How long is the longest list in the hash table?

Q: What items are in it?

Hashing with Chaining

Table should look like this:



A: Longest list contains 3 items and is in slot 2

It contains 12, 7, 2 ordered in reverse order of insertion since we add to front of list

Closed Hashing

An alternative to chaining is ***closed hashing***

The idea is to store elements only in the table itself

No external storage such as linked lists etc

We need a different way to resolve collisions

The basic idea of closed hashing insertion:

- Look at slot given by hash function, insert if empty
- If not, systematically probe the table until an empty slot is found

Closed hashing schemes are classified by the way they systematically probe the table

The simplest method is ***linear probing***

Closed Hashing

Probing effectively adds another parameter to the hash function:

$$h : U \times \{0, 1, \dots, m - 1\} \rightarrow \{0, 1, \dots, m - 1\}$$

The idea behind closed hashing:

- First we try slot: $h(k, 0)$
- If this is already occupied, we try $h(k, 1)$
- Then $h(k, 2)$ and so on
 $\langle h(k, 0), h(k, 1), \dots, h(k, m - 1) \rangle$

The sequence:

$$\{0, 1, \dots, m - 1\}$$

will be a permutation of

In other words, we will try every slot in the table eventually

Closed Hashing

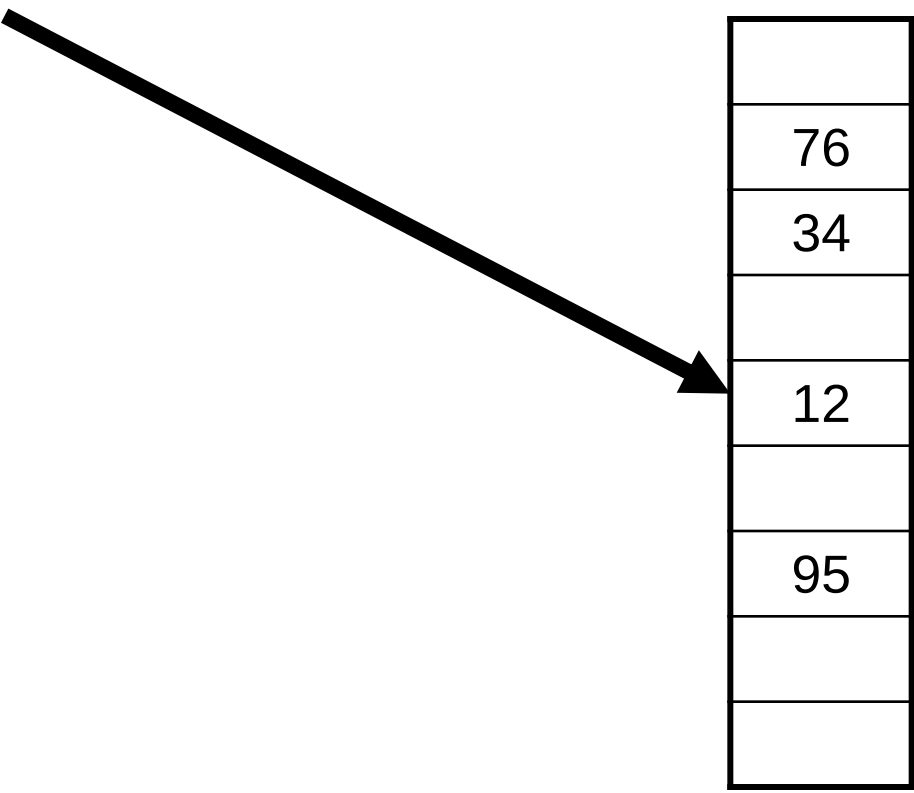
Example: insert $k = 63$

76
34
12
95

Closed Hashing

Example: insert $k = 63$

1. Probe $h(63,0)$



collision

Closed Hashing

Example: insert $k = 63$

1. Probe $h(63,0)$

2. Probe $h(63,1)$



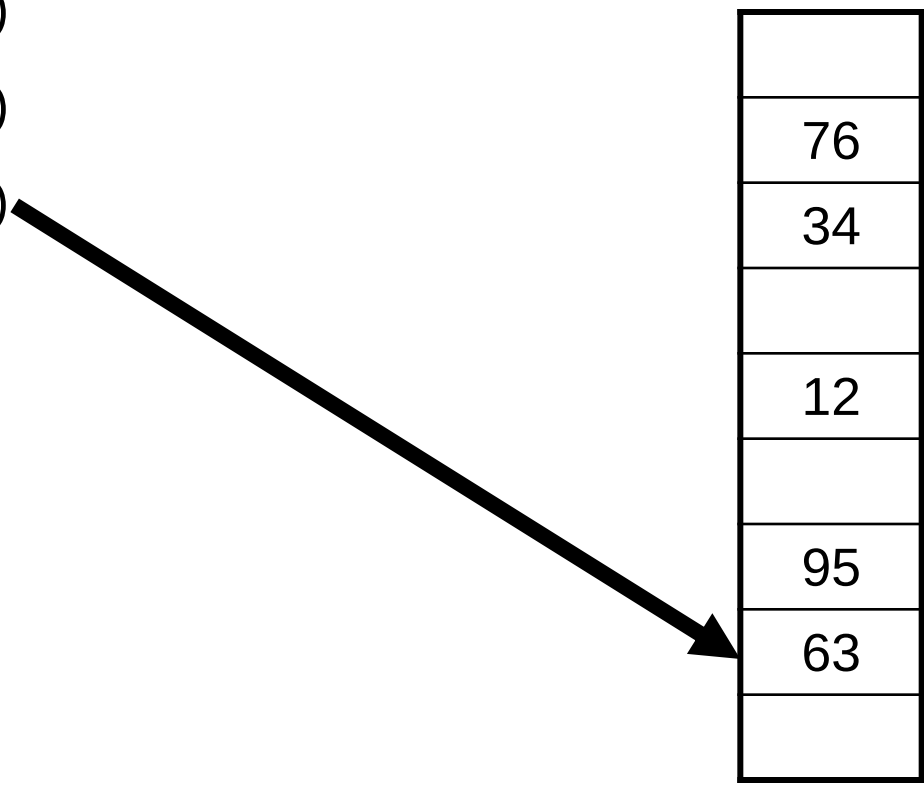
76
34
12
95

collision

Closed Hashing

Example: insert $k = 63$

- 1. Probe $h(63,0)$
- 2. Probe $h(63,1)$
- 3. Probe $h(63,2)$



insertion

Closed Hashing

Q: How do we retrieve an item from a closed hash table? At what point can we say the item is not in the table?

Closed Hashing

Q: How do we retrieve an item from a closed hash table? At what point can we say the item is not in the table?

A: To retrieve key k :

1. Set i to 0
2. If slot $h(k,i) = \text{NULL}$, item not in table - stop
3. If key of item in slot $h(k,i) = k$, then return item and stop
4. Otherwise, increase i (if $i=m$, set i to 0)

Closed Hashing

Q: How do we retrieve an item from a closed hash table? At what point can we say the item is not in the table?

A: To retrieve key k :

1. Set i to 0
2. If slot $h(k,i) = \text{NULL}$, item not in table - stop
3. If key of item in slot $h(k,i) = k$, then return item and stop
4. Otherwise, increase i (if $i=m$, set i to 0)

Q: What check is missing from this algorithm?

Closed Hashing

Q: How do we retrieve an item from a closed hash table? At what point can we say the item is not in the table?

A: To retrieve key k :

1. Set i to 0
2. If slot $h(k,i) = \text{NULL}$, item not in table - stop
3. If key of item in slot $h(k,i) = k$, then return item and stop
4. Otherwise, increase i (if $i=m$, set i to 0)

Q: What check is missing from this algorithm?

A: If the table is full but does not contain the item, we will go on forever.
We need to check if $h(k,i)=h(k,0)$ for $i>0$
If it does, we've looked at the whole table

Closed Hashing

Q: How do we delete an item from a closed hash table?

Closed Hashing

Q: How do we delete an item from a closed hash table?

A: We can't just set it to NULL, otherwise items after it in the probe sequence will never be found

Instead, deleted items are marked as ***ghost*** records

Closed Hashing

Q: How do we delete an item from a closed hash table?

A: We can't just set it to NULL, otherwise items after it in the probe sequence will never be found

Instead, deleted items are marked as ***ghost*** records

For retrieval:

if we encounter a ghost record, we ignore it and keep searching

Closed Hashing

Q: How do we delete an item from a closed hash table?

A: We can't just set it to NULL, otherwise items after it in the probe sequence will never be found

Instead, deleted items are marked as ***ghost*** records

For retrieval:

if we encounter a ghost record, we ignore it and keep searching

Q: What effect will ghost records have on insertion?

Closed Hashing

Q: How do we delete an item from a closed hash table?

A: We can't just set it to NULL, otherwise items after it in the probe sequence will never be found

Instead, deleted items are marked as ***ghost*** records

For retrieval:

if we encounter a ghost record, we ignore it and keep searching

Q: What effect will ghost records have on insertion?

A: We can now insert into either a free or ghosted space, so deletions will eventually be overwritten

Linear Probing

The simplest closed hashing strategy is ***linear probing***

Given our original hash function, h , we define linear probing as:

$$h'(k, i) = (h(k) + i) \bmod m$$

- If a slot is occupied, we look at the next slot, then the next and so on.
- If we go past m , we wrap round to zero.
- If i reaches m , we have looked at the whole table and there are no free slots, return an overflow error

Linear Probing

The main drawback of linear probing is ***primary clustering***

This is where long runs of occupied slots build up, increasing average search time

E.g. consider a hash table for surnames where the hash function is the alphabetic value of the first letter

Many surnames begin with 'S' and will hash to the same slot

Data will cluster after the S slot, leading to a long run of items that must be searched through

These long runs will get in the way of an increasing number of other hash locations

Long runs will eventually join together

Linear Probing

...

P	
Q	
R	
S	Smithers
T	Smiffy
U	Smitherington
V	Smivers
W	Smithe
X	Schmidt

...

Linear Probing

Exercise: insert the following keys into a closed hash table using linear probing:

9, 2, 4, 7, 12

$$h(k) = k \bmod 5$$

using the hash function:

and storing the items in the following array:

0	
1	
2	
3	
4	

Q: What is the contents of the array after the insertions?

Linear Probing

A: Insert: **9**, 2, 4, 7, 12

$$9 \bmod 5 = 4$$

0	
1	
2	
3	
4	9

insertion

Linear Probing

A: Insert: 9, **2**, 4, 7, 12

$$2 \bmod 5 = 2$$

0	
1	
2	2
3	
4	9

insertion

Linear Probing

A: Insert: 9, 2, **4**, 7, 12

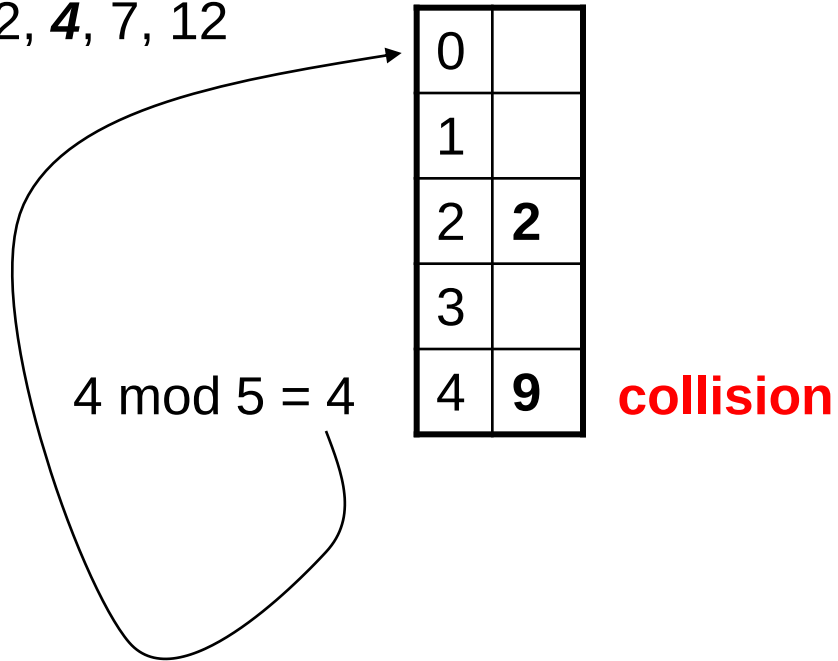
$$4 \bmod 5 = 4$$

0	
1	
2	2
3	
4	9

collision

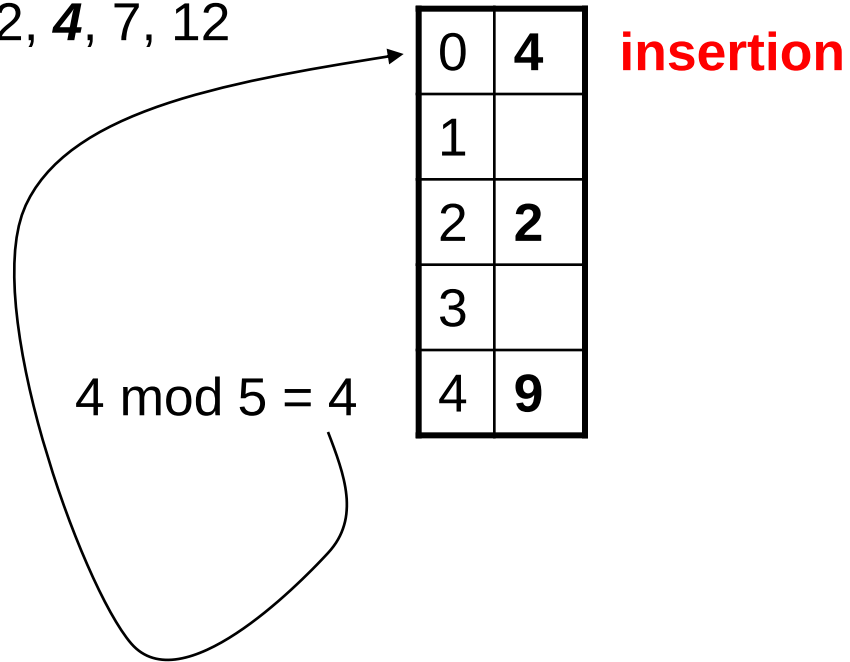
Linear Probing

A: Insert: 9, 2, **4**, 7, 12



Linear Probing

A: Insert: 9, 2, **4**, 7, 12



Linear Probing

A: Insert: 9, 2, 4, **7**, 12

$$7 \bmod 5 = 2$$

0	4
1	
2	2
3	
4	9

collision

Linear Probing

A: Insert: 9, 2, 4, **7**, 12

$$7 \bmod 5 = 2$$



0	4
1	
2	2
3	7
4	9

insertion

Linear Probing

A: Insert: 9, 2, 4, 7, **12**

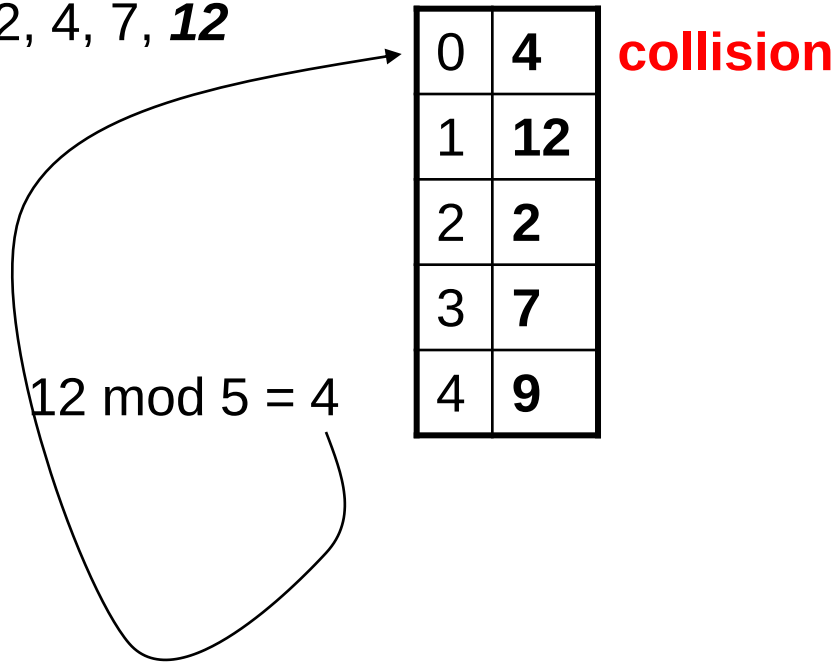
$$12 \bmod 5 = 4$$

0	4
1	
2	2
3	7
4	9

collision

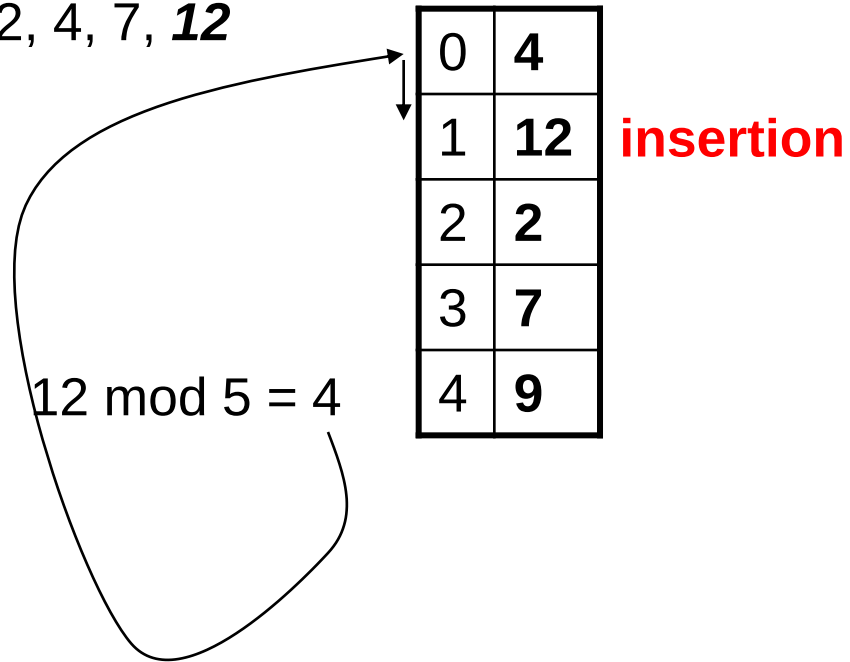
Linear Probing

A: Insert: 9, 2, 4, 7, **12**



Linear Probing

A: Insert: 9, 2, 4, 7, **12**



Quadratic Probing

An alternative to linear probing is ***quadratic probing***

The idea is to replace the linear increment by a quadratic function, e.g.:

$$h'(k, i) = (h(k) + ci^2) \bmod m$$

A more complex function than ci^2 can be used

We need to ensure that we will try every slot as part of our probe sequence, otherwise an insertion may fail when the table isn't full

To ensure this, the hash table size must be prime

Keys that hash to the same slot will still follow the same probe sequence, leading to ***secondary clustering***

However, **long linear runs will be avoided**, meaning other hash slots won't be unevenly disrupted

Double Hashing

An even better alternative is double hashing

This combines two hash functions to make a probe sequence:

$$h'(k, i) = (h_1(k) + i \cdot h_2(k)) \bmod m$$

In other words, the first hash function determines our starting slot, the second determines the probe sequence

As for quadratic probing we need to be careful to ensure all slots are examined

This means $h_2(k)$ must be relatively prime to m (share no common positive factors)

An easy way to do this is make m a power of 2 and design $h_2(k)$ to only produce an odd numbered output

In practice, double hashing works very well

Efficiency of Closed Hashing

Analysing closed hashing schemes precisely is complex

Suffice to say that the average case complexity of insert, delete, retrieve is close to $\Theta(1)$

E.g. if the table is 90% full, the expected number of probes is 10

Performance will deteriorate as the load factor increases

The choice between hashing with chaining and closed hashing comes down to practical issues

E.g. linear probing has ***locality of reference*** (adjacent slots are examined) this is fast using caches etc

Design factors will determine which is best

Rehashing

The probing strategies given above tend to give good performance until the table approaches being full

After this, performance can decrease rapidly

One way around this is that when our hash table is sufficiently full that performance is suffering, we ***rehash***

This means allocating a larger table, choosing a new hash function and reinserting all elements from the old table into the new

This will also clean up ghost records that are in the table

However, rehashing is expensive

Some examples of hash functions

Q: What makes a good hash function?

Some examples of hash functions

Q: What makes a good hash function?

A: A good hash function will approximately satisfy the simple uniform hashing condition

We want an approximately even distribution of keys throughout the table

Some examples of hash functions

Q: What makes a good hash function?

A: A good hash function will approximately satisfy the simple uniform hashing condition

We want an approximately even distribution of keys throughout the table

The difficulty is that obtaining an even distribution on average requires knowing the probability distribution of the keys in advance

Some examples of hash functions

Q: What makes a good hash function?

A: A good hash function will approximately satisfy the simple uniform hashing condition

We want an approximately even distribution of keys throughout the table

The difficulty is that obtaining an even distribution on average requires knowing the probability distribution of the keys in advance

Example: insert the following keys:

FILE0000.CHK, FILE0001.CHK, FILE0002.CHK, ...

Some examples of hash functions

Q: What makes a good hash function?

A: A good hash function will approximately satisfy the simple uniform hashing condition

We want an approximately even distribution of keys throughout the table

The difficulty is that obtaining an even distribution on average requires knowing the probability distribution of the keys in advance

Example: insert the following keys:

FILE0000.CHK, FILE0001.CHK, FILE0002.CHK, ...

Using first letter would be terrible

Using 4 digit number mod m would be a good choice

Lesson: what is “good” depends totally on the input

See Cormen 11.3 for some hash functions used in practice

Conclusions

We should now know:

- What a hash table is, collisions
- Chaining versus Closed Hashing
- Linear probing, quadratic probing, double hashing, rehashing
- Hash tables are messy but practical, effectively $O(1)$ for dictionary operations

Next time:

- Another useful data structure: Priority queues

Recommended reading for this lecture:

- Mehlhorn Chapter 4
- Cormen Chapters 11
- Skiena 3.7